

Práctico: Elemento Mayoritario

Asignatura:	Programación III	Paradigma:	Divide y Conquista
Tema:	Búsqueda del Elemento Mayoritario	Lenguaje:	Java

1. Objetivos Didácticos

- Aplicar el paradigma de **Divide y Conquista** para resolver problemas de búsqueda eficientes.
- Implementar algoritmos recursivos en **Java** estructurando la lógica de forma limpia y mantenible.
- Verificar analítica y experimentalmente la complejidad temporal $O(n)$ alcanzada frente a enfoques tradicionales.

2. Contexto Teórico y Definición

Dado un arreglo de enteros A de tamaño n , se define como **Elemento Mayoritario** a aquel valor que aparece estrictamente más de $n / 2$ veces en el arreglo.

Importante: Un arreglo puede contener a lo sumo un solo elemento mayoritario. Sin embargo, no todos los arreglos poseen uno. Por ejemplo, en `[1, 2, 1, 1, 3, 1, 1]` el elemento `1` aparece 5 veces de 7 ($5 > 3.5$), por lo cual es mayoritario. En cambio, en `[1, 2, 3, 4, 5]` no existe elemento mayoritario.

3. Consigna del Ejercicio

Se solicita desarrollar una solución en Java siguiendo la especificación del algoritmo de Divide y Conquista estudiado en la asignatura:

3.1. Estructura de la Clase `ElementoMayoritario`

1. **Método Recursivo** `buscarCandidato(int[] v, int inicio, int fin)` :
 - Agrupa elementos adyacentes en pares.
 - Si los dos elementos del par son iguales, conserva uno de ellos para la siguiente etapa de reducción.
 - Maneja adecuadamente el caso de sub-arreglos con un número impar de elementos según la especificación teórica.
 - Retorna un entero candidato a elemento mayoritario (o `null` si no hay candidato posible).
2. **Método Principal** `obtenerElementoMayoritario(int[] v)` :
 - Invoca a `buscarCandidato` para obtener la propuesta de elemento.
 - Si obtiene un candidato, realiza una segunda pasada lineal en $O(n)$ sobre el arreglo original para verificar su frecuencia real.

- Si la frecuencia contabilizada es estrictamente mayor a $n/2$, retorna un `Optional<Integer>` con dicho valor. En caso contrario, retorna `Optional.empty()`.

4. Casos de Prueba Requeridos

En el método `main` de su programa, verifique la ejecución utilizando al menos los siguientes casos de prueba:

Caso	Vector de Entrada	Resultado Esperado
Caso 1	[1, 2, 1, 1, 3, 1, 1]	Elemento Mayoritario = 1
Caso 2	[1, 2, 3, 4, 5, 6]	No existe elemento mayoritario
Caso 3	[2, 2, 2, 4, 5, 6]	No existe elemento mayoritario (Frecuencia = 3/6, no supera $n/2$)

5. Cuestionario de Análisis (Para la Entrega)

Responda brevemente a las siguientes cuestiones teóricas junto con el código fuente de su solución:

1. **Ecuación de Recurrencia:** Plantee la ecuación de recurrencia $T(n)$ del método `buscarCandidato` y explique justificadamente por qué la complejidad temporal total resulta ser $\Theta(n)$.
2. **Efectos Secundarios (Side Effects):** ¿Por qué es fundamental operar sobre una copia del arreglo original al ejecutar el algoritmo de reducción en Java? Explique las consecuencias sobre el estado del vector en memoria.